ETHALNETHRA MART

Online Payment Issue Report

Money is deducted from the customer bank account, but the backend does not receive a success signal and the order is not confirmed.

Field	Value
App	ethalnethra_ecommerce_app (Flutter)
API base	https://api.app.ethalnethra.com
Gateway	Razorpay Payment Links (in-app WebView)
Date	18 August 2026
App version	1.0.0+12
Question	Can this be fixed with backend-only change (no app update)?
Answer	YES for order creation (webhook). App success screen can also work if callback URL matches existing Flutter URL checks.

Bottom line: Bank debit = Razorpay captured the payment. That is not the same as “order placed”. The Flutter app never calls a verify/place API after online pay. Backend must confirm payment via Razorpay webhook. An app update is not required to stop missing orders.

1. Problem statement

After online checkout, the customer is charged (amount leaves the bank account) but:

- Backend does not get / save a payment-success event.
- Order is missing or still pending in the database.
- App may or may not show “Order accepted”.
- Need to know if this is Flutter, backend, or both — and whether a Play Store update is required.

2. Current APIs used by the app

Defined in **lib/core/api/api_endpoints.dart**. Base URL in **lib/core/api/api_config.dart**.

Method	Path	When used	Creates real order?
POST	/api/orders/checkout	Checkout sheet preview (totals + rewards)	No
POST	/api/orders/create-payment	User taps Place Order with ONLINE	No (only Razorpay link)
POST	/api/orders/place	COD instant cart only	Yes
POST	/api/orders/place-preorder	COD pre-order only	Yes

Method	Path	When used	Creates real order?
GET	/api/orders	Orders list after checkout refresh	Read
GET	/api/orders/{id}	Track Order (uses razorpay_order_id after online)	Read
—	verify-payment / confirm-payment / webhook handler in app	NOT implemented in Flutter	Missing

Critical gap: *After Razorpay collects money, Flutter does not call* `/api/orders/place` *and does not call any verify API. There is no webhook listener in the app (webhooks belong on the server).*

3. Request / response data the app already parses

3.1 Create payment — POST /api/orders/create-payment

Request body (Flutter):

```
{
  "address_id": 123,
  "type": "instant" | "preorder",
  "use_reward_coins": true | false
}
```

Response fields parsed into CreatePaymentResponse:

JSON field	Dart field	Notes
success	success	bool
key	key	Razorpay key (unused by WebView flow)
razorpay_order_id	razorpayOrderId	Later misused as order_id on success screen
payment_link	paymentLink	Opened in WebView
payment_link_id	paymentLinkId	Stored in memory only; never sent back
amount	amount	Integer (paise likely)
currency	currency	Default INR
type	type	instant / preorder
reward.*	reward	used, available_coins, can_redeem, reward_discount, payable_amount
customer.*	customer	name, email, contact
summary.*	summary	subtotal, delivery_charge, reward_discount, total

3.2 Place order — POST /api/orders/place or /place-preorder (COD only)

```
{
  "address_id": 123,
  "payment_method": "COD",
  "use_reward_coins": true | false
}
```

PlaceOrderResponse fields:

JSON field	Dart field
order_id / orderId / id / razorpay_order_id	orderId
order_number / orderNumber	orderNumber
status	status
tracking_url	trackingUrl
dispatch_date	dispatchDate
notes	notes
delivery_charge	deliveryCharge
total / summary.total / reward.payable_amount	total
discount	discount
loyalty_applied	loyaltyApplied
eta	eta
payment_method	paymentMethod
message	message

4. Current Flutter flow

4.1 COD (works)

```
Checkout sheet
  -> POST /api/orders/place  OR  /api/orders/place-preorder
  -> Backend creates real order
  -> OrderSuccessScreen with real order_id / order_number
```

4.2 ONLINE (broken)

```
Checkout sheet  (checkout_bottom_sheet.dart)
  -> POST /api/orders/create-payment
  -> Open PaymentWebViewScreen(paymentLink)
  -> Watch WebView URL for success/failure STRINGS
  -> On guessed success: BUILD A FAKE PlaceOrderResponse LOCALLY
  -> markOnlinePaymentComplete(fake)
  -> Show OrderSuccessScreen
  -> NEVER notify backend that payment succeeded
```

4.3 How Flutter guesses success (fragile)

File: **lib/views/cart/payment_webview_screen.dart** — methods `_isSuccessUrl / _isFailureUrl / _maybeComplete`.

Type	URL contains (case-insensitive)
Success	payment_id=
Success	razorpay_payment_id
Success	status=success
Success	payment/success
Success	paymentsuccess

Type	URL contains (case-insensitive)
Success	/success
Success	rzp AND success
Failure	status=failed / failure / cancelled / canceled
Failure	payment/failed , paymentfailed , /failed

As soon as a URL matches, the WebView is closed (`Navigator.pop`). If that happens before Razorpay redirects to the backend callback URL, the server never receives the browser redirect. Money is already captured.

4.4 Fake success object built in the app

```
PlaceOrderResponse(
  orderId: payment.razorpayOrderId,      // NOT backend order id
  orderNumber: payment.razorpayOrderId,  // NOT EN-xxxx
  status: 'paid',                        // assumed, not verified
  total: summary.total,
  paymentMethod: 'ONLINE',
)
```

Track Order then calls `GET /api/orders/{razorpay_order_id}`. If backend only looks up internal order_id, tracking fails even after webhook creates the order.

5. Source files

File	Role
lib/views/cart/widgets/checkout_bottom_sheet.dart	COD vs ONLINE branch; opens WebView
lib/views/cart/payment_webview_screen.dart	Razorpay WebView; URL guess; fake success
lib/core/providers/orders_action_providers.dart	createOnlinePayment; markOnlinePaymentComplete
lib/core/repositories/orders_repository.dart	createPayment + placeOrder HTTP calls
lib/core/api/api_endpoints.dart	API paths
lib/core/api/api_config.dart	https://api.app.ethalnethra.com
lib/core/models/orders/orders_models.dart	CreatePaymentResponse, PlaceOrderResponse
lib/core/models/safe_json_parse.dart	Loose JSON parsing for payment payloads
lib/views/cart/order_success_screen.dart	Shows accepted + Track Order

6. Root cause — Flutter vs backend

This is **both sides**. Razorpay charges the card before your server confirms the order. The server only learns about success if (1) webhook, (2) Flutter verify API, or (3) Razorpay redirect callback finishes.

What you observe	Likely cause	Side
Razorpay Paid, DB has no order / still pending	Webhook missing, failing, or not creating order	Backend
WebView closes instantly, no callback in server logs	Flutter matched URL and popped too early	Flutter
App shows Order accepted, orders list empty	Fake local PlaceOrderResponse	Flutter

What you observe	Likely cause	Side
App never shows success, money gone	URL matcher missed; user closed WebView; no verify	Flutter + Backend
User paid then killed the app	Redirect cannot run; only webhook can save the order	Backend

7. Can this be fixed with backend-only change? (NO APP UPDATE)

YES — for the money-taken / order-missing issue. The current app never tells the server “paid”. The server must learn it from Razorpay.

Problem	App update needed?	Backend-only fix
Amount gone, order missing in DB	NO	Razorpay webhook payment_link.paid + payment.captured
Success not saved on server	NO	Webhook + payment-link callback_url
User paid then closed WebView	NO	Webhook only (redirect will not run)
App success screen	Mostly NO	Callback URL must contain strings Flutter already checks
Track Order after online pay	NO if compatible	GET /api/orders/{id} must also find by razorpay_order_id
Perfect UX: verify + poll + real order id	YES later	Needs Flutter change; not required to stop missing orders

7.1 Backend changes that work with the CURRENT app

- **Webhook (must have).** Events: payment_link.paid and payment.captured. Verify Razorpay signature. Find pending record by payment_link_id. Create / mark order paid. Idempotent (same event twice = same order).
- **Store pending data on create-payment.** Save payment_link_id, razorpay_order_id, address_id, cart snapshot, amount, type, user_id. Webhook needs this to create the order later.
- **Set callback_url on the Razorpay payment link** to e.g. https://api.app.ethalnethra.com/api/orders/payment-callback. After pay, WebView loads this URL so backend can confirm from query params.
- **Match existing Flutter URL checks** so the current app shows success without an update. Include razorpay_payment_id and/or status=success and/or /success in the callback URL.
- **Track Order compatibility.** Flutter uses razorpay_order_id as order_id. GET /api/orders/{id} should also look up by razorpay_order_id, not only internal id.

Example callback URL the CURRENT app will treat as success:

```
https://api.app.ethalnethra.com/api/orders/payment-callback?status=success&razorpay_payment_id=pay_xxx&razorpay_payment
```

7.2 What backend cannot fully fix without a later app update

- If Flutter matches /success too early and kills the WebView before callback loads, redirect never hits you. Webhook still saves the order.
- App may show success a few seconds before webhook finishes; orders list can be empty until refresh.
- App still does not call a verify API — you cannot add that without shipping a new build.
- Fake local success object (razorpay_order_id as order number) stays until Flutter is updated.

8. Recommended target architecture

```
Flutter App                Your Backend                Razorpay
    |                           |                          |
    | POST /create-payment      |                          |
    |-------------------------->| Create pending record    |
    |                           | Create payment link      |
    |                           |------------------------->|
```

```
| payment_link + ids         |                            |
|<---------------------------|                            |
| Open WebView (link)        |                            |
|-------------------------------------------------------->|
|                            |                            | Charge customer
|                            |     webhook paid           |
|                            |<---------------------------|
|                            | Verify signature           |
|                            | Create / confirm ORDER     |
|  (optional) callback URL   |                            |
|  WebView loads callback    | Confirm if not yet done    |
|  URL matches success       |                            |
|  Show success screen       |                            |
| GET /api/orders            | Return real order          |
|<---------------------------|                            |
```

9. Suggested backend API contracts

9.1 POST /api/orders/create-payment (already exists — extend)

Keep the same request/response so the current app keeps working. Internally: create pending payment/order, create Razorpay payment link with callback_url, store mapping.

9.2 Razorpay webhook (new — required)

Endpoint example: `POST /api/webhooks/razorpay`. Do not require the Flutter auth token. Verify `X-Razorpay-Signature`. Handle `payment_link.paid` and `payment.captured`.

9.3 GET/POST payment-callback (new — helps current app UI)

Razorpay Payment Links typically redirect with:

- razorpay_payment_id
- razorpay_payment_link_id
- razorpay_payment_link_status=paid
- razorpay_signature

Verify signature, confirm order if webhook has not already, return a simple HTML page whose URL still contains status=success and razorpay_payment_id so Flutter pops success.

9.4 POST /api/orders/verify-payment (optional, for a future app update)

```
Request:
{
  "payment_link_id": "plink_xxx",
  "razorpay_payment_id": "pay_xxx"   // optional if only link id is known
}

Response (idempotent — safe if webhook already created the order):
{
  "success": true,
  "order_id": "123",
  "order_number": "EN-1001",
  "status": "paid",
  "total": 499,
  "payment_method": "ONLINE"
}

Pending (webhook still in flight):
{ "success": false, "status": "pending", "message": "Payment received, order confirming" }
```

10. Future Flutter changes (not required to ship this fix)

- Do not invent PlaceOrderResponse locally.

bullet Detect paid from callback query (razorpay_payment_link_status=paid or razorpay_payment_id).
bullet Call verify-payment; show success only if a real order_id is returned.
bullet If pending, poll 10–20 seconds (webhook may be slightly late).
bullet If still pending: “Payment received. Order will appear shortly.” Do not say accepted.
bullet Do not pop WebView on a random /success inside a Razorpay domain URL. Wait for your callback.

11. How to verify after backend deploy (no new app)

#	Check	Pass if
1	Razorpay Dashboard → Payments / Payment Links	Status Paid for the test payment
2	Razorpay → Webhooks → deliveries	payment_link.paid returns HTTP 2xx
3	Backend DB / admin orders	Real order exists, status paid, linked to payment_link_id
4	Backend logs	Webhook and/or callback received after pay
5	Current app success screen	WebView redirected to callback URL containing razorpay_payment_id
6	Current app Track Order	GET /api/orders/{razorpay_order_id} returns the order
7	Kill app immediately after pay	Webhook still creates the order (user sees it on next open)
8	Replay same webhook	No duplicate order (idempotent)

12. Implementation checklist (backend team)

Priority	Task	Needed for current app?
P0	Razorpay webhook endpoint + signature verify	Yes
P0	On create-payment: persist pending order/payment mapping	Yes
P0	On webhook paid: create/confirm order, clear cart server-side	Yes
P0	Idempotency on payment_id / payment_link_id	Yes
P1	Set callback_url on payment link create	Yes (for in-app success UI)
P1	Callback URL includes status=success and razorpay_payment_id	Yes (matches Flutter)
P1	GET /api/orders/{id} lookup by razorpay_order_id too	Yes (Track Order)
P2	verify-payment API for next Flutter release	No (future)
P2	Refund / failed-after-capture handling	Yes (ops)

13. Decision

Do not wait for an app update to stop this production issue. Ship webhook + pending record + callback_url + razorpay_order_id lookup. That fixes: amount gone from bank, success not stored on backend, order missing.

Ship a Flutter update later for: real order_id on success screen, verify + poll, safer URL matching, no fake success.

Document generated from the Ethalnethra Flutter codebase analysis on 18 August 2026. No backend repository was available; API behaviour is inferred from the app client.